DS 203 – Big Data Essentials
PL/pgSQL: Control Structures & User-Defined Functions

Control Structures

Control structures in PL/pgSQL let you control the flow of execution inside your procedural code. Think of them as the "decision-making" and "repetition" tools that let SQL act more like a programming language.

IF Statements

Purpose: Decide what to do based on conditions.

Key Points:

- Run code only if a certain condition is true.
- Combine with ELSE/ELSIF for multiple conditions.
- Works with comparisons, ranges, set checks, or boolean expressions.

Example: We want to know if the active customer base is large, medium, or normal.

```
DO $$
DECLARE
   cust_active_count INT;
BEGIN
   SELECT COUNT(*) INTO cust_active_count
   FROM customer
   WHERE active = 1;

   IF cust_active_count > 1000 THEN
      RAISE NOTICE 'Large active customer base: %', cust_active_count;
         ELSIF cust_active_count > 500 THEN
      RAISE NOTICE 'Medium active customer base: %', cust_active_count;
   ELSE
      RAISE NOTICE 'Active customers: %', cust_active_count;
   END IF;
END; $$;
```

Explanation:

- COUNT(*) INTO cust_active_count stores the result of the query in a variable.
- The IF checks if that count is over 1000 or between 500 – 1000 and prints a message accordingly.

CASE Statements

Purpose: Handle multiple possible outcomes more cleanly than multiple IF...ELSIF.

Types:

1. **Simple CASE** – compares one value against different matches.
2. **Searched CASE** – uses independent boolean expressions.

Example: Categorizing the most recent rental rate into *Low*, *Medium*, or *High*

```
DO $$
DECLARE
   rate NUMERIC;
   category VARCHAR;
BEGIN
   SELECT rental_rate INTO rate
   FROM rental
   JOIN inventory USING (inventory_id)
   JOIN film USING (film_id)
   ORDER BY rental_id DESC
   LIMIT 1;

   CASE
      WHEN rate < 2 THEN category := 'Low';
      WHEN rate BETWEEN 2 AND 4 THEN category := 'Medium';
      ELSE category := 'High';
   END CASE;

   RAISE NOTICE 'Most recent rental rate: %, Category: %', rate, category;
END; $$;
```

Explanation:

- We pull the most recent rental's rental_rate.
- The CASE assigns a label based on price ranges.

FOR Loops

Purpose: Iterate over rows or ranges.

Types:

- FOR ... IN SELECT – loop through query results.
- FOR counter IN start..end – loop through numeric ranges.

Example: Listing all long films

```
DO $$
DECLARE
   rec RECORD; --generic row holder
         counter INTEGER := 0;
BEGIN
   FOR rec IN
      SELECT film_id, title FROM film WHERE length > 120
   LOOP
                  counter := counter +1;
      RAISE NOTICE '%. Long film: % (ID: %)', counter, rec.title, rec.film_id;
   END LOOP;
END; $$;
```

Explanation:

- rec RECORD; stores each row in the loop.
- The loop runs once for every film longer than 120 minutes.

WHILE Loops

Purpose: Repeat code while a condition is true.

Example: Simple counter example with CONTINUE and EXIT

```
DO $$
DECLARE
   counter INT := 1;
BEGIN
   WHILE counter <= 5 LOOP
      IF counter = 3 THEN
         counter := counter + 1;
         CONTINUE;
      END IF;
      RAISE NOTICE 'Counter is %', counter;
      IF counter = 4 THEN
         EXIT;
      END IF;
      counter := counter + 1;
   END LOOP;
END; $$;
```

Explanation:

- CONTINUE skips the rest of the current loop and moves to the next iteration.
- EXIT stops the loop entirely.

User-Defined Functions

A function is reusable code you can call in SQL statements.

Benefits:

- Encapsulation: Put logic in one place and reuse it.
- Maintainability: Change function logic without updating multiple queries.
- Performance: Reduce network trips by doing more on the server.

Creating a Scalar Function

Purpose: Return a single value (number, text, date, etc.).

Example – Count the number of films between two lengths:

```
CREATE OR REPLACE FUNCTION get_film_count(len_from INT, len_to INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
   film_count INT;
BEGIN
   SELECT COUNT(*) INTO film_count
   FROM film
   WHERE length BETWEEN len_from AND len_to;

   RETURN film_count;
END; $$;
```

Usage:

```
SELECT get_film_count(60, 90);
```

Explanation:

- RETURNS INT tells PostgreSQL the output type.
- Parameters len_from and len_to act like variables you pass when calling the function.
- RETURN sends the result back.

Functions Returning Tables

Purpose: Return multiple rows and columns.

Example – List rentals from a specific store:

```
CREATE OR REPLACE FUNCTION get_rentals_by_store(storeid INT)
RETURNS TABLE(rent_id INT, cust_id SMALLINT, rent_date TIMESTAMP)
LANGUAGE plpgsql
AS $$
BEGIN
   RETURN QUERY
   SELECT rental_id, customer_id, rental_date
   FROM rental
   JOIN inventory USING (inventory_id)
   WHERE store_id = storeid;
END; $$;
```

Usage:

```
SELECT * FROM get_rentals_by_store(1);
```

Explanation:

- RETURNS TABLE(...) defines the shape of the result.
- RETURN QUERY directly outputs rows from a SELECT.

Dropping a Function

To drop a function, specify its name and parameter types. If the function has a unique name, it is not mandatory to specify parameter type, but it is still a good practice to follow:

```
DROP FUNCTION get_film_count(INT, INT);
```

Function Parameter Modes

PostgreSQL supports three main parameter modes which determine how data flows between the caller and the function:

Mode	Direction of Data Flow	Purpose
IN	Caller → Function	Pass a value into the function (default mode).
OUT	Function → Caller	Return a value from the function as an output parameter.
INOUT	Both ways	Pass a value in, modify it inside the function, and return it.

IN Parameters:

- **Default mode** (you can omit IN keyword).
- You pass a value to the function; the function can use it but **cannot change it for the caller**.

Example:

```
CREATE OR REPLACE FUNCTION get_customer_fullname(IN cust_id INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
   fullname TEXT;
BEGIN
   SELECT first_name || ' ' || last_name
   INTO fullname
   FROM customer
   WHERE customer_id = cust_id;

   RETURN fullname;
END; $$;
```

Usage:

```
SELECT get_customer_fullname(1);
```

OUT Parameters:

- Used when you want to **return values without RETURN**. Especially, when you want to return multiple values.
- The output variable is declared in the function signature and **automatically returned**.

Example:

```
CREATE OR REPLACE FUNCTION get_film_stats(len_from INT, len_to INT, OUT
film_count INT)
LANGUAGE plpgsql
AS $$
BEGIN
   SELECT COUNT(*) INTO film_count
   FROM film
   WHERE length BETWEEN len_from AND len_to;
END; $$;
```

Usage:

```
SELECT get_film_stats(60, 90);
```

Notice that film_count is not **explicitly returned** – PostgreSQL does it automatically.

INOUT Parameters:

- The same parameter works as both input and output.
- Useful when you want to modify a value passed into the function and return it.

Example:

```
CREATE OR REPLACE FUNCTION add_days_to_date(INOUT rental_date DATE,
days_to_add INT)
LANGUAGE plpgsql
AS $$
BEGIN
   rental_date := rental_date + days_to_add;
END; $$;
```

Usage:

```
SELECT add_days_to_date('2020-01-01', 5);
```

-- Returns: 2020-01-06

- Here rental_date starts with the value passed in, gets modified, and is returned.

References:

ChatGPT. (2025). *Guide on PL/pgSQL control structures and user-defined functions with dvdrental examples* [AI-generated teaching material]. OpenAI. https://chat.openai.com/